Laboratorio 15

Control de Calidad de las soluciones

Objetivos

- Anticipar y gestionar errores comunes durante el desarrollo de soluciones.
- Emplear técnicas básicas de depuración.

Conceptos calve

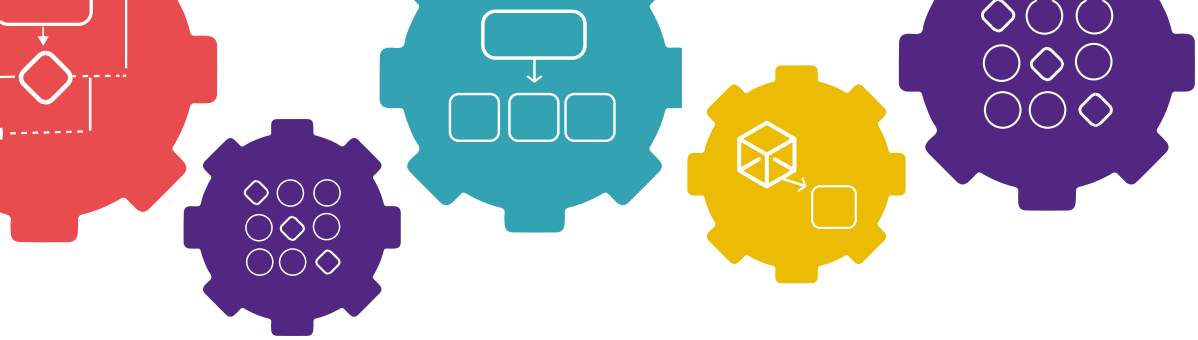
Defecto:

Es una falla en la solución que ocasiona un comportamiento equivocado. Existen varios tipos de defectos:

- De escritura
- Mala gramática y ambigüedad
- Inconsistencias
- Defectos lógicos y matemáticos.

Los defectos pueden ser detectados y eliminados en cualquier etapa del proceso de desarrollo, pero cuanto antes se encuentren será mejor. Algunos defectos aparecerán solo cuando la solución se ha implementado. En estos casos se requieren de pruebas (testing) y seguimiento paso a paso (debugging).

> Pruebas de calidad: Verifican que el sistema desarrollado se comporta como se espera e identifica áreas específicas de fallo.



Seguimiento paso a paso: Enfoque sistemático que permite encontrar el error mientras el programa se ejecuta. Se emplea la descomposición para corregir comportamiento inesperados en tiempo real.

Ejercicios

1. **Tipos de Defectos:** Seleccione la opción que mejor describa el tipo de defecto presentado.

a. Un programa en C# para la recopilación de datos personales muestra el siguiente mensaje en pantalla:

"Ingresar su fecha de nacimiento y presionar Enter: "

Como resultado, el sistema ha sido incapaz de interpretar las respuestas de los usuarios debido a que todas vienen en formatos distintos (ej. DD/MM/YYYY, MM-DD-YYYY, YYYY-MM-DD, etc.)

- Defecto de Ambigüedad
- Defecto de Inconsistencia
- Defecto Lógico
- Defecto matemático

El mensaje no especifica el formato de la fecha, por lo que los usuarios lo interpretan de maneras distintas.

b. Se ha desarrollado un programa que calcula la suma, resta producto y cociente entre dos números ingresados por el usuario. Sin embargo, el programa presenta defectos cuando el usuario ingresa 0 en ambos valores pues no existe la división entre cero.

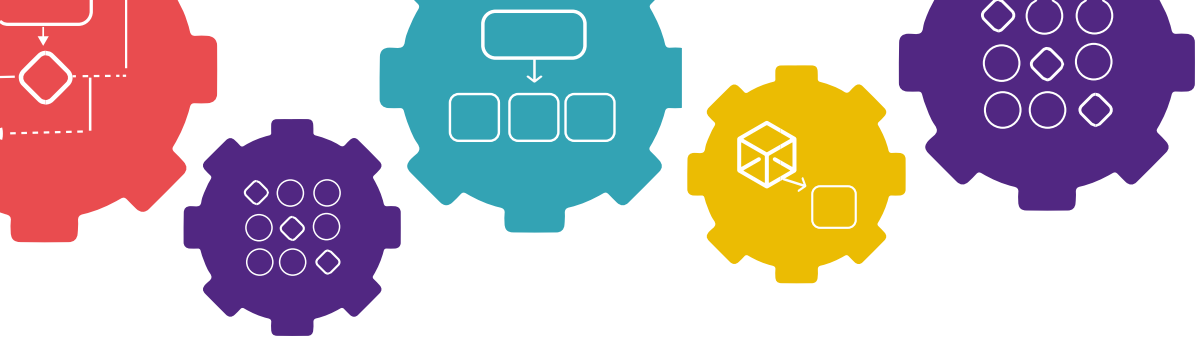
- Defecto de Escritura
- Defecto matemático
- Defecto de Ambigüedad
- Defecto de Inconsistencia.

No se valida la división entre cero, una operación matemáticamente indefinida.

c. Se solicita desarrollar un programa que genere 20 números pares aleatorios entre 50 y 100. Deberá mostrar el listado de números en pantalla, resaltando en color cyan los números primos, y calcular las medidas de tendencia central.

- Defecto de Ambigüedad
- Defecto de Escritura
- Defecto matemático
- Defecto de Inconsistencia.

ningún número par mayor a 2 puede ser primo



d. Un sistema bancario en línea requiere que la contraseña de sus usuarios contenga entre 8 y 16 caracteres. A pesar de que se implementó una validación recientemente, se han detectado múltiples incidencias de fallo.

```
if (longitud >= 8 || longitud <= 16) {
        AceptarContraseña(contraseña)
} else{
        RechazarContraseña(contraseña)
}
```

- Defecto de Ambigüedad
- Defecto de Escritura
- Defecto lógico
- Defecto de Inconsistencia.

El operador || (OR) acepta cualquier longitud. Debe ser && (AND) para que se cumpa.

2. <u>Pruebas de calidad (testing) – bottom up</u>:

Elabore un plan de pruebas para validar el correcto funcionamiento de los siguientes métodos. Para cada método deberá realizar tres pruebas, indicando:

- Descripción de la prueba
- Valores de entrada
- Resultado Esperado
- Resultado obtenido, si el método cumple o no la prueba.

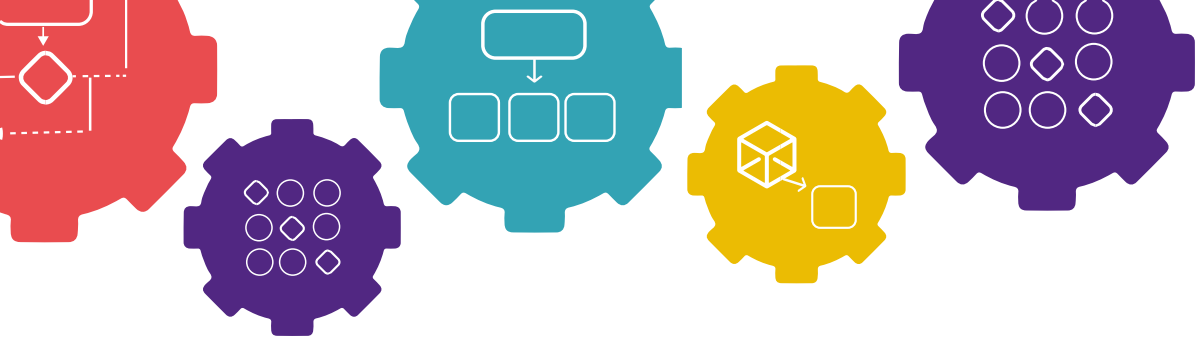
El objetivo del ejercicio es evidenciar, si los hay, los defectos en cada método y sugerir una propuesta de mejora.

Ejemplo: El método CalcularPromedio( ) recibe tres notas entre 0 y 100 y retorna el promedio de las tres.

```
0 referencias
static double CalcularPromedio(double nota1, double nota2, double nota3) {
    return (nota1 + nota2 + nota3) / 3;
}
```

Caso	Entrada	Resultado Esperado	Resultado Obt.
Notas dentro del rango	70, 80, 90	80	Aprobado
Nota Negativa	-10, 80, 90	Error o Rechazo	Reprobada
Nota mayor a 100	120, 80, 90	Error o Rechazo	Reprobada

Añadir: if (montoRetirado <= 0 || montoRetirado > saldo)



Propuesta: Añadir una validación que determine si las entradas se encuentran en el rango de 0 a 100 antes de realizar los cálculos.

a. El método RetirarEfectivo( ) permite retirar dinero de una cuenta bancaria por medio de un cajero automático. Esta recibe dos parámetros; El saldo disponible en la cuenta y el monto que desea retirar.

```
0 referencias
static void RetirarEfectivo(ref double saldo, double montoRetirado) {
    saldo = saldo - montoRetirado;
}
```

Caso	Entrada	Resultado Esperado	Resultado Obt.
Retiro valido	saldo =100, monto= 300	saldo = 700	aptobado saldo 700
retiro igual al saldo	saldo = 500 monto = 500	saldo =0	aprobado saldo 0
retiro mayor al saldo	saldo = 200 monto =500	error	rechazado saldo = -300
monto negativo	saldo = 500 monto = -100	error	rechazado saldo = 600

b. La función CalcularDescuento( ) permite aplicar un descuento o rebaja al precio original de un producto. Para esto utiliza dos entradas; el precio original del producto y el porcentaje de descuento, entre 0 y 100, que debemos aplicar.

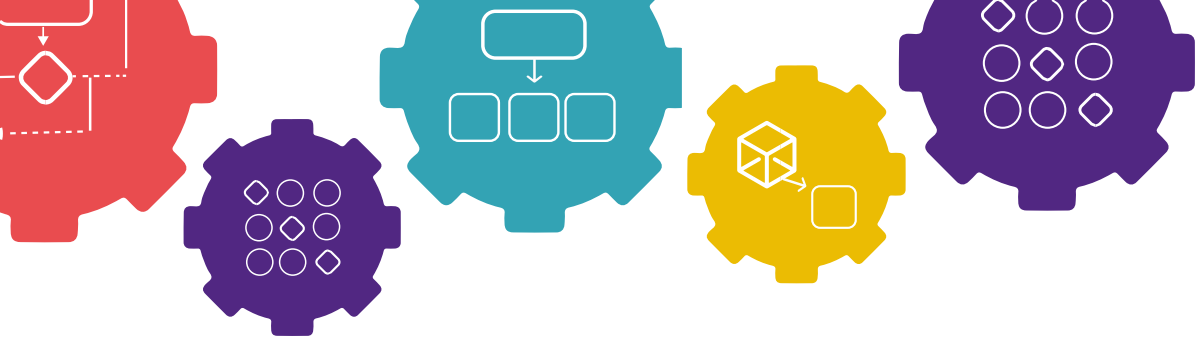
```
0 referencias
static double CalcularDescuento(double precio, double porcentaje)
{
    return precio - (precio * porcentaje / 100);
}
```

Caso	Entrada	Resultado Esperado	Resultado Obt.
descuento valido	precio = 200 porcentaje =25	150	arpobado = 150
descuento 0%	precio = 100 procentaje=0	100	aprobado = 100
descuento 100%	precio = 100 procentaje=100	0	aprobado = 0
porcentaje negativo	precio = 100 procentaje=-20	error	reprobado = 120

alidar precio > 0 y porcentaje >= 0 && porcentaje <= 100 antes de calcular.

c. El método Depositar( ) permite aumentar el saldo de una cuenta bancaria. Este recibe como entrada el saldo disponible del cliente y el monto que desea depositar.

```
0 referencias
static void Depositar(ref double saldo, double monto)
{
    saldo = saldo + monto;
}
```



Caso	Entrada	Resultado Esperado	Resultado Obt.
deposito valido	saldo = 500 monto = 200	saldo = 700	aprobado saldo = 700
deposito de 0	saldo = 500 monto = 0	error	rechazado saldo = 500
monto negativo	saldo = 500 monto = -100	error	rechazado saldo = 400
saldo negativo	saldo =-100 monto =100	error	rechado saldo = 0

Añadir: if (monto <= 0) rechazar antes de actualizar el saldo.

3. **<u>Seguimiento paso a paso (Debugging)</u>:**

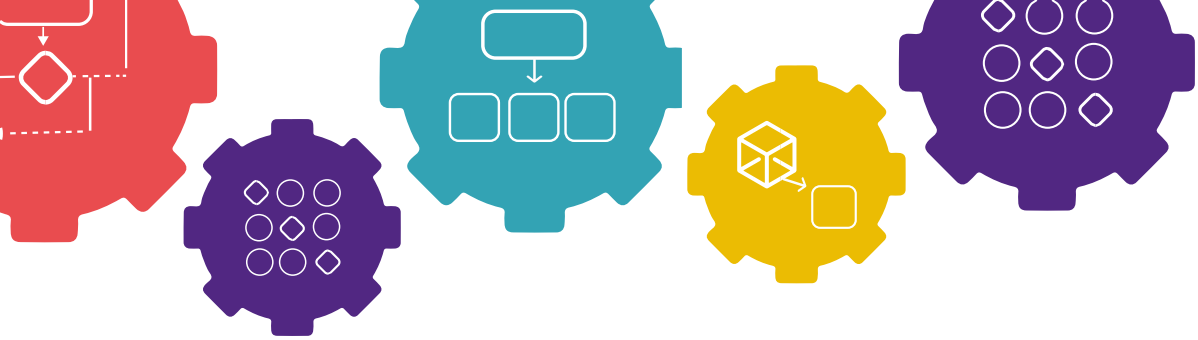
Crear un proyecto de consola en C# e ingresar el código proporcionado para simular el pago de un crédito personal.

```
static void Main()
{
    double capital = 1000;
    double tasa = 0.05;
    double intereses = 0;
    double abonos = 0;

    for (int mes = 1; mes <= 12 && capital > 0; mes++)
    {
        // Cálculo de intereses del mes
        intereses = capital * tasa;

        // Abono realizado cada mes
        abonos = 100 + (mes * 10);

        // Actualización del capital
        capital = capital + intereses - abonos;
    }
}
```

Deberá utilizar herramientas de depuración (*debugging*) mediante el uso de **breakpoints** para observar cómo cambian las variables durante la ejecución del programa.

<u>Instrucciones</u>:

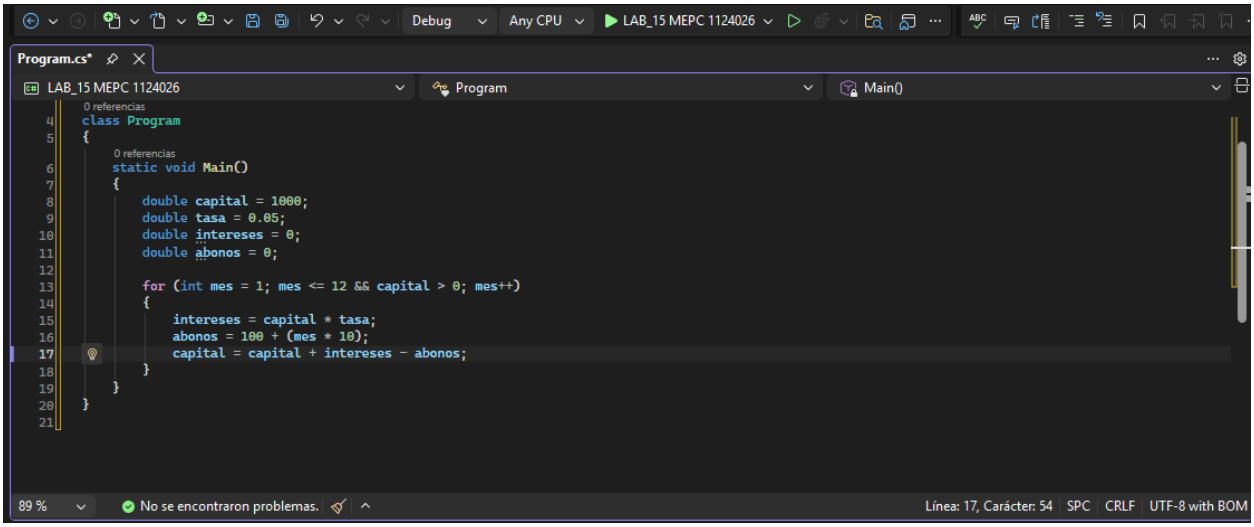
a. Ejecute el programa en modo depuración (*Debug*).
b. Coloque un **breakpoint** dentro del ciclo for, específicamente en la línea donde se actualiza el capital.
c. Utilice las herramientas de depuración:
   - Step Over (F10)
   - Ventana de variables locales
d. Observe cómo cambian las siguientes variables durante cada iteración:
   - capital
   - intereses
   - abonos

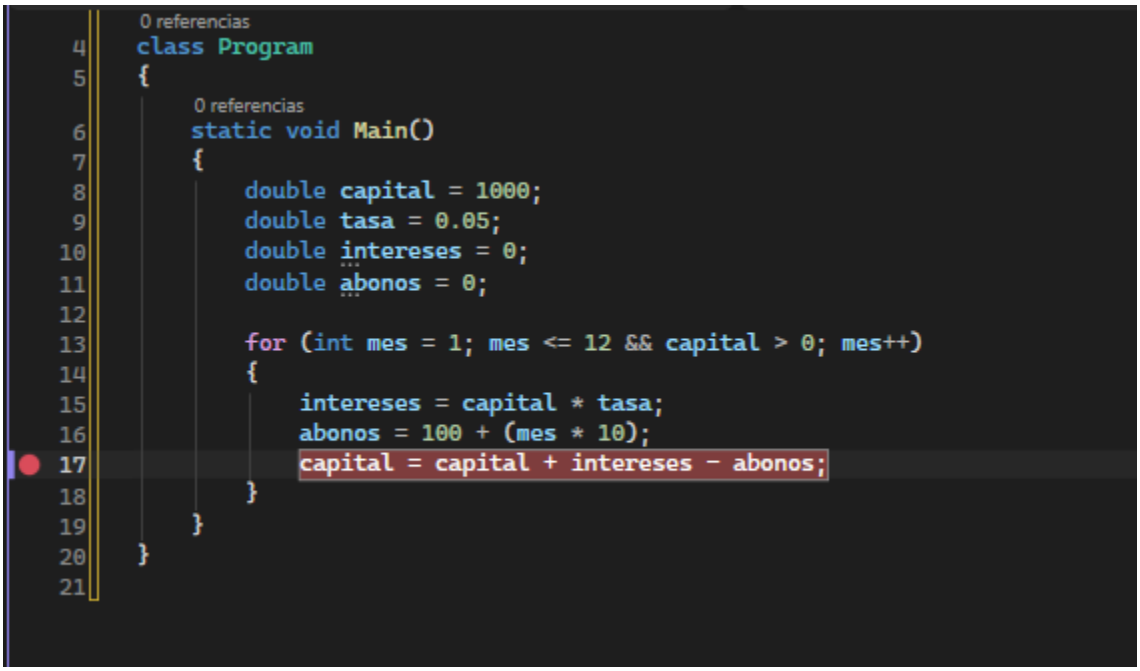


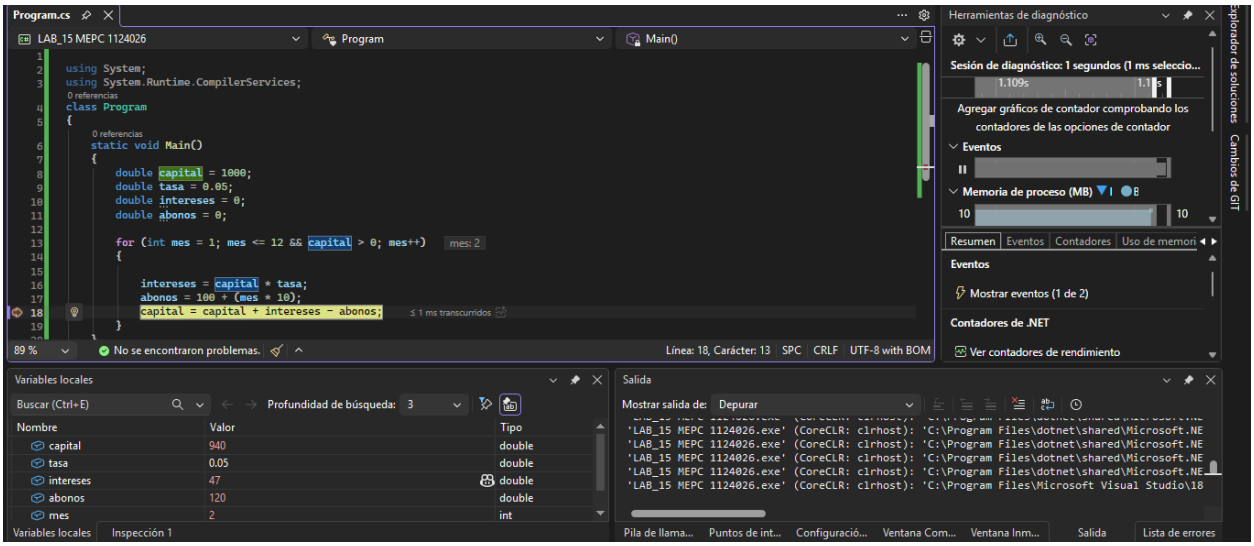
- tasa
- mes

e. Tome capturas de pantalla donde se visualicen claramente los valores de las variables al finalizar:
   - Mes 2
   - Mes 4
   - Mes 6
   - Mes 8

Ejercicio	Criterio	Puntos	Total
Ejercicio 1: Tipos de Defectos	Inciso A	5 pts	**20 ptos**
	Inciso B	5 pts	
	Inciso C	5 pts	
	Inciso D	5 pts	
Ejercicio 2: Testing	Diseño de pruebas adecuados, no faltan escenarios.	10 pts	**40 ptos**
	Identifica correctamente todos los defectos presentes en las funciones	10 pts	
	Documenta de forma clara los resultados esperados y el comportamiento observado en cada prueba.	10 pts	
	Propone validaciones o correcciones adecuadas para solucionar los defectos encontrados	10 pts	
Ejercicio 3: Debugging	Estado de las variables en el Mes 2	10 pts	**40 ptos**
	Estado de las variables en el Mes 4	10 pts	
	Estado de las variables en el Mes 6	10 pts	
	Estado de las variables en el Mes 8	10 pts	
Total			**100 ptos**


Debug
Any CPU
LAB_15 MEPC 1124026
Program.cs*
LAB_15 MEPC 1124026
Program
Main()
0 referencias
class Program
{
0 referencias
static void Main()
{
double capital = 1000;
double tasa = 0.05;
double intereses = 0;
double abonos = 0;
for (int mes = 1; mes <= 12 && capital > 0; mes++)
{
intereses = capital * tasa;
abonos = 100 + (mes * 10);
capital = capital + intereses - abonos;
}
}
}
89 %
No se encontraron problemas.
Línea: 17, Carácter: 54
SPC
CRLF
UTF-8 with BOM


0 referencias
class Program
{
0 referencias
static void Main()
{
double capital = 1000;
double tasa = 0.05;
double intereses = 0;
double abonos = 0;
for (int mes = 1; mes <= 12 && capital > 0; mes++)
{
intereses = capital * tasa;
abonos = 100 + (mes * 10);
capital = capital + intereses - abonos;
}
}
}


Program.cs
LAB_15 MEPC 1124026
Program
Main()
using System;
using System.Runtime.CompilerServices;
0 referencias
class Program
{
0 referencias
static void Main()
{
double capital = 1000;
double tasa = 0.05;
double intereses = 0;
double abonos = 0;
for (int mes = 1; mes <= 12 && capital > 0; mes++)
mes: 2
{
intereses = capital * tasa;
abonos = 100 + (mes * 10);
capital = capital + intereses - abonos;
≤ 1 ms transcurridos
}
89 %
No se encontraron problemas.
Línea: 18, Carácter: 13
SPC
CRLF
UTF-8 with BOM
Herramientas de diagnóstico
Sesión de diagnóstico: 1 segundos (1 ms seleccio...
1.109s
Agregar gráficos de contador comprobando los contadores de las opciones de contador
Eventos
Memoria de proceso (MB)
10
10
Resumen
Eventos
Contadores
Uso de memori
Eventos
Mostrar eventos (1 de 2)
Contadores de .NET
Ver contadores de rendimiento
Explorador de soluciones
Cambios de GIT
Variables locales
Buscar (Ctrl+E)
Profundidad de búsqueda: 3
Nombre
Valor
Tipo
capital
940
double
tasa
0.05
double
intereses
47
double
abonos
120
double
mes
2
int
Variables locales
Inspección 1
Salida
Mostrar salida de: Depurar
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NE
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NE
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NE
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NE
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\Microsoft Visual Studio\18
Pila de llama...
Puntos de int...
Configuració...
Ventana Com...
Ventana Inm...
Salida
Lista de errores

Program.cs — LAB_15 MEPC 1124026 — Program — Main()

```
using System;
using System.Runtime.CompilerServices;
0 referencias
class Program
{
    0 referencias
    static void Main()
    {
        double capital = 1000;
        double tasa = 0.05;
        double intereses = 0;
        double abonos = 0;

        for (int mes = 1; mes <= 12 && capital > 0; mes++)        mes: 4
        {

            intereses = capital * tasa;
            abonos = 100 + (mes * 10);
            capital = capital + intereses - abonos;        ≤ 1 ms transcurridos
        }
```

89 % — No se encontraron problemas. — Línea: 18, Carácter: 13 — SPC — CRLF — UTF-8 with BOM

Herramientas de diagnóstico — Sesión de diagnóstico: 1 segundos (1 ms seleccio... — 1.109s — Agregar gráficos de contador comprobando los contadores de las opciones de contador — Eventos — Memoria de proceso (MB) — Resumen | Eventos | Contadores | Uso de memori — Eventos: Mostrar eventos (3 de 4) — Contadores de .NET: Ver contadores de rendimiento

Variables locales — Buscar (Ctrl+E) — Profundidad de búsqueda: 3

Nombre	Valor	Tipo
capital	780.35	double
tasa	0.05	double
intereses	39.017500000000005	double
abonos	140	double
mes	4	int

Salida — Ocultar automáticamente — Depurar

```
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NE
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NE
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NE
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\Microsoft Visual Studio\18
El subproceso '.NET TP Worker' (17864) terminó con código 0 (0x0).
```

Program.cs — LAB_15 MEPC 1124026 — Program — Main()

```
using System;
using System.Runtime.CompilerServices;
0 referencias
class Program
{
    0 referencias
    static void Main()
    {
        double capital = 1000;
        double tasa = 0.05;
        double intereses = 0;
        double abonos = 0;

        for (int mes = 1; mes <= 12 && capital > 0; mes++)        mes: 6
        {

            intereses = capital * tasa;
            abonos = 100 + (mes * 10);
            capital = capital + intereses - abonos;        ≤ 1 ms transcurridos
        }
```

89 % — No se encontraron problemas. — Línea: 18, Carácter: 13 — SPC — CRLF — UTF-8 with BOM

Herramientas de diagnóstico — Sesión de diagnóstico: 1 segundos (1 ms seleccio... — 1.11s — Agregar gráficos de contador comprobando los contadores de las opciones de contador — Eventos — Memoria de proceso (MB) — Resumen | Eventos | Contadores | Uso de memori — Eventos: Mostrar eventos (4 de 6) — Contadores de .NET: Ver contadores de rendimiento

Variables locales — Buscar (Ctrl+E) — Profundidad de búsqueda: 3

Nombre	Valor	Tipo
capital	563.3358750000001	double
tasa	0.05	double
intereses	28.166793750000007	double
abonos	160	double
mes	6	int

Salida — Mostrar salida de: Depurar

```
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\Microsoft Visual Studio\18
El subproceso '.NET TP Worker' (17864) terminó con código 0 (0x0).
El subproceso '.NET TP Worker' (12960) terminó con código 0 (0x0).
El subproceso '.NET TP Worker' (8364) terminó con código 0 (0x0).
El subproceso '.NET TP Worker' (8280) terminó con código 0 (0x0).
```

Program.cs — LAB_15 MEPC 1124026 — Program — Main()

```
using System;
using System.Runtime.CompilerServices;
0 referencias
class Program
{
    0 referencias
    static void Main()
    {
        double capital = 1000;
        double tasa = 0.05;
        double intereses = 0;
        double abonos = 0;

        for (int mes = 1; mes <= 12 && capital > 0; mes++)        mes: 8
        {

            intereses = capital * tasa;
            abonos = 100 + (mes * 10);
            capital = capital + intereses - abonos;        ≤ 1 ms transcurridos
        }
```

89 % — No se encontraron problemas. — Línea: 18, Carácter: 13 — SPC — CRLF — UTF-8 with BOM

Herramientas de diagnóstico — Sesión de diagnóstico: 1 segundos (1 ms seleccio... — .11s — Agregar gráficos de contador comprobando los contadores de las opciones de contador — Eventos — Memoria de proceso (MB) — Resumen | Eventos | Contadores | Uso de memori — Eventos: Mostrar eventos (1 de 8) — Contadores de .NET: Ver contadores de rendimiento

Variables locales — Buscar (Ctrl+E) — Profundidad de búsqueda: 3

Nombre	Valor	Tipo
capital	283.07780218750014	double
tasa	0.05	double
intereses	14.153890109375007	double
abonos	180	double
mes	8	int

Salida — Mostrar salida de: Depurar

```
'LAB_15 MEPC 1124026.exe' (CoreCLR: clrhost): 'C:\Program Files\Microsoft Visual Studio\18
El subproceso '.NET TP Worker' (17864) terminó con código 0 (0x0).
El subproceso '.NET TP Worker' (12960) terminó con código 0 (0x0).
El subproceso '.NET TP Worker' (8364) terminó con código 0 (0x0).
El subproceso '.NET TP Worker' (8280) terminó con código 0 (0x0).
```